

Practical Insights into Data Analysis and Machine Learning

PROGRAM - 4

Develop a program to load the Iris dataset. Implement the k-Nearest Neighbors (k-NN) algorithm for classifying flowers based on their features. Split the dataset into training and testing sets and evaluate the model using metrics like accuracy and F1-score. Test it for different values of k (e.g., $k=1,3,5$) and evaluate the accuracy. Extend the k-NN algorithm to assign weights based on the distance of neighbors (e.g., $weight=1/d^2$). Compare the performance of weighted k-NN and regular k-NN on a synthetic or real-world dataset.

Objective

To implement and evaluate the k-Nearest Neighbors (k-NN) algorithm for classification problems. It aims to compare the performance of regular and weighted k-NN models based on accuracy and F1-score.

4. Introduction

The k-Nearest Neighbors (k-NN) algorithm is a simple yet powerful supervised learning technique used for classification and regression tasks. This program focuses on implementing k-NN for classifying flowers from the Iris dataset based on their features. The dataset consists of three species of Iris flowers—Setosa, Versicolor, and Virginica—and has four numerical attributes: sepal length, sepal width, petal length, and petal width.

This program will explore both standard and weighted k-NN, where weights are assigned based on the inverse squared distance of neighbors ($1/d^2$). The dataset is split into training and testing sets to evaluate the model's performance using accuracy and F1-score. By testing different values of k (e.g., 1, 3, and 5), we analyze the impact of neighbourhood size on classification performance. Additionally, we compare the effectiveness of weighted k-NN against the regular k-NN model on other datasets. This analysis provides insights into the role of distance weighting in improving classification accuracy.

4.1 k-Nearest Neighbors (k-NN)

The k-Nearest Neighbors (k-NN) algorithm is a simple yet powerful supervised learning technique used for classification and regression. It is a non-parametric and instance-based learning algorithm, means, it makes predictions based on the stored training instances without assuming any underlying data distribution.

Working of k-NN

1. **Choose a value for k :** The number of nearest neighbors to consider for classification.
2. **Compute the distance:** For a given query (test) instance, calculate its distance to all training instances using Euclidean distance,

$$d(p, q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3. **Sort the distance** in an ascending order and **select the k nearest neighbors** (training instances) closest to the test instance.
4. Predict the class of the test instance by **Majority voting** (for classification), i.e., the class label that appears most frequently among the k neighbors is assigned to the test instance.
5. **Weighted k-NN (optional):** Instead of simple majority voting, assign weights to neighbors based on their distance (e.g., inverse squared distance: $1/d^2$) giving closer neighbors more influence.
6. **Output the prediction:** The test instance is classified based on the majority vote or weighted vote.

4.2 Classifying using k-NN

1. Consider the training Dataset

Instance	CGPA	CIE	Project Evaluation	Result
1	9.2	82	8	Pass
2	3	79	7	Pass
3	8.5	82	8	Pass
4	6	45	5	Fail
5	6.5	55	4	Fail
6	8.2	71	7	Pass
7	5.8	40	5	Fail
8	8.9	91	9	Pass

2. Consider a Test Instance: predict the performance of a new student with:

CGPA = 7.0, CIE = 60, Project Evaluation = 6

3. Compute distances to all training instances using Euclidean distance from the new student

$$d = \sqrt{(CGPA_{test} - CGPA_{train})^2 + (Assessment_{test} - Assessment_{train})^2 + (Project_{test} - Project_{train})^2}$$

Below is the table with distance values calculated for the test instance (CGPA=7.0, CIE=60, Project Evaluation=6) using the Euclidean distance formula:

Instance	CGPA	CIE	Project Evaluation	Result	Euclidean Distance
1	9.2	82	8	Pass	22.20
2	3	79	7	Pass	19.44
3	8.5	82	8	Pass	22.14
4	6	45	5	Fail	15.06
5	6.5	55	4	Fail	5.40
6	8.2	71	7	Pass	11.11
7	5.8	40	5	Fail	20.06
8	8.9	91	9	Pass	31.20

Here is the sorted table based on the Euclidean Distance values in ascending order:

Instance	CGPA	CIE	Project Evaluation	Result	Euclidean Distance
5	6.5	55	4	Fail	5.48
6	8.2	71	7	Pass	11.11
4	6	45	5	Fail	15.06
2	3	79	7	Pass	19.44
7	5.8	40	5	Fail	20.06
3	8.5	82	8	Pass	22.14
1	9.2	82	8	Pass	22.20
8	8.9	91	9	Pass	31.27

4. Interpretation of Results

The classification is based on the majority voting of the closest neighbors.

- $k=1 \rightarrow$ Instance 5 (Fail)
Test instance is classified as *Fail*
- $k=3$ Nearest Neighbors $\rightarrow$ Instances 5 (Fail), 6 (Pass), and 4 (Fail)
Test instance is classified as *Fail*
- $k=5$ Nearest Neighbors $\rightarrow$ Instances 5 (Fail), 6 (Pass), 4 (Fail), 2 (Pass), and 7 (Fail)
Test instance is classified as *Fail*

4.3 Application of k-NN

1. Healthcare & Medical Diagnosis

- Disease Prediction: Used to classify diseases based on patient symptoms, medical history, and test results.
- Cancer Detection: Helps in detecting breast cancer by classifying tumor cells as benign or malignant.
- Diabetes Risk Prediction: Predicts whether a patient is at risk of diabetes based on health metrics.

2. Recommender Systems

- Movie & Music Recommendations: Used in streaming services like Netflix and Spotify to suggest content based on user preferences.
- E-commerce: Amazon and Flipkart use k-NN to recommend products based on purchase history.

3. Image & Video Recognition

- Face Recognition: Used in security systems and social media (Facebook's auto tagging feature).
- Handwritten Digit Recognition: Used in postal services to recognize handwritten ZIP codes on letters.

4. Fraud Detection & Cybersecurity

- Credit Card Fraud Detection: Identifies unusual transactions based on historical spending behavior.
- Intrusion Detection Systems (IDS): Used to detect malicious activities in network traffic.

5. Finance & Stock Market Prediction

- Loan Approval Systems: Banks use k-NN to assess whether a customer is eligible for a loan.
- Stock Price Prediction: Classifies stock trends based on past market data.

6. Agriculture & Environmental Science

- Crop Disease Detection: Helps farmers predict diseases in crops using leaf images.
- Weather Forecasting: Classifies weather conditions based on historical data.

4.4 Program

```
# import the libraries

import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score

# Load the Iris dataset
iris = load_iris()
X = iris.data # Features
y = iris.target # Labels

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Function to evaluate k-NN for different values of k
def cls_knn(X_train, X_test, y_train, y_test, k_values, weighted=False):
    results = {}
    for k in k_values:
        if weighted:
            # Weighted k-NN
            knn = KNeighborsClassifier(n_neighbors=k, weights='distance')
        else:
            # Regular k-NN
            knn = KNeighborsClassifier(n_neighbors=k, weights='uniform')

        # Train the model
        knn.fit(X_train, y_train)

        # Make predictions
        y_pred = knn.predict(X_test)

        # Calculate accuracy and F1-score
        accuracy = accuracy_score(y_test, y_pred)

        # Weighted F1-score for multi-class
        f1 = f1_score(y_test, y_pred, average='weighted')
        results[k] = {'accuracy': accuracy, 'f1_score': f1}
    return results
```

```

# Test for different values of k
k_values = [1, 3, 5]

# Evaluate regular k-NN
print("Regular k-NN Results:")
regular_results = cls_knn(X_train, X_test, y_train, y_test, k_values, weighted=False)
for k, metrics in regular_results.items():
    print(f"k={k}: Accuracy={metrics['accuracy']:.2f}, F1-Score={metrics['f1_score']:.2f}")

Regular k-NN Results:
k=1: Accuracy=1.00, F1-Score=1.00
k=3: Accuracy=1.00, F1-Score=1.00
k=5: Accuracy=1.00, F1-Score=1.00

# Evaluate weighted k-NN
print("\nWeighted k-NN Results:")
weighted_results = cls_knn(X_train, X_test, y_train, y_test, k_values, weighted=True)
for k, metrics in weighted_results.items():
    print(f"k={k}: Accuracy={metrics['accuracy']:.2f}, F1-Score={metrics['f1_score']:.2f}")

Weighted k-NN Results:
k=1: Accuracy=1.00, F1-Score=1.00
k=3: Accuracy=1.00, F1-Score=1.00
k=5: Accuracy=1.00, F1-Score=1.00

# Compare performance
print("\nComparison of Regular k-NN and Weighted k-NN:")
for k in k_values:
    regular_acc = regular_knn[k]['accuracy']
    weighted_acc = weighted_knn[k]['accuracy']
    print(f"k={k}: Regular k-NN Accuracy={regular_acc:.2f}, Weighted k-NN Accuracy={weighted_acc:.2f}")

Comparison of Regular k-NN and Weighted k-NN:
k=1: Regular k-NN Accuracy=1.00, Weighted k-NN Accuracy=1.00
k=3: Regular k-NN Accuracy=1.00, Weighted k-NN Accuracy=1.00
k=5: Regular k-NN Accuracy=1.00, Weighted k-NN Accuracy=1.00

```

Looking at these results showing 100% accuracy and F1-score for regular k-NN with k=1, k=3, and k=5, this perfect performance (accuracy=1.00, F1-score=1.00) is likely due to one of several reasons:

1. **Dataset splitting issue:** The random state used for train_test_split might have created a particularly "easy" test set where the class boundaries are very clear and samples are easily classified.
2. **Small test set:** The test set is very small, perfect classification becomes more likely by chance.
3. **Iris dataset characteristics:** The Iris dataset has a clean structure for two of its three classes. Specifically, Setosa is completely linearly separable from the other classes.

Versicolor and Virginica have some overlap but can often be well-separated with proper scaling

4. **Overfitting:** $k=1,3,5$ is prone to overfitting as it simply returns the class of the nearest neighbor. Perfect test set performance might indicate that the model is memorizing rather than generalizing.

To get more meaningful comparative results between regular and weighted k-NN,

- Use k-fold cross-validation instead of a single train/test split
- Try a more challenging dataset like Wine or Breast Cancer
- Add some noise to the data to make the classification task more difficult
- Use a different random state for the train/test split

Program using Wine Dataset

```
# import the libraries
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score

# from sklearn.datasets import load_wine
wine = load_wine()
X = wine.data
y = wine.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```

# Function to evaluate k-NN for different values of k
def cls_knn(X_train, X_test, y_train, y_test, k_values, weighted=False):
    results = {}
    for k in k_values:
        if weighted:
            # Weighted k-NN
            knn = KNeighborsClassifier(n_neighbors=k, weights='distance')
        else:
            # Regular k-NN
            knn = KNeighborsClassifier(n_neighbors=k, weights='uniform')

        # Train the model
        knn.fit(X_train, y_train)

        # Make predictions
        y_pred = knn.predict(X_test)

        # Calculate accuracy and F1-score
        accuracy = accuracy_score(y_test, y_pred)

        # Weighted F1-score for multi-class
        f1 = f1_score(y_test, y_pred, average='weighted')
        results[k] = {'accuracy': accuracy, 'f1_score': f1}
    return results

# Test for different values of k
k_values = [1, 3, 5]

# Evaluate regular k-NN
print("Regular k-NN Results:")
regular_knn = cls_knn(X_train, X_test, y_train, y_test, k_values, weighted=False)
for k, metrics in regular_knn.items():
    print(f"k={k}: Accuracy={metrics['accuracy']:.4f}, F1-Score={metrics['f1_score']:.4f}")

Regular k-NN Results:
k=1: Accuracy=0.7963, F1-Score=0.7954
k=3: Accuracy=0.7407, F1-Score=0.7424
k=5: Accuracy=0.7407, F1-Score=0.7424

# Evaluate weighted k-NN
print("\nWeighted k-NN Results:")
weighted_knn = cls_knn(X_train, X_test, y_train, y_test, k_values, weighted=True)
for k, metrics in weighted_knn.items():
    print(f"k={k}: Accuracy={metrics['accuracy']:.4f}, F1-Score={metrics['f1_score']:.4f}")

```


Weighted k-NN Results:

k=1: Accuracy=0.7963, F1-Score=0.7954

k=3: Accuracy=0.7407, F1-Score=0.7424

k=5: Accuracy=0.7593, F1-Score=0.7593

```
# Compare performance
print("\nComparison of Regular k-NN and Weighted k-NN:")
for k in k_values:
    regular_acc = regular_knn[k]['accuracy']
    weighted_acc = weighted_knn[k]['accuracy']
    print(f"k={k}: Regular k-NN Accuracy={regular_acc:.4f}, Weighted k-NN Accuracy={weighted_acc:.4f}")
```

Comparison of Regular k-NN and Weighted k-NN:

k=1: Regular k-NN Accuracy=0.7963, Weighted k-NN Accuracy=0.7963

k=3: Regular k-NN Accuracy=0.7407, Weighted k-NN Accuracy=0.7407

k=5: Regular k-NN Accuracy=0.7407, Weighted k-NN Accuracy=0.7593

Analysis of Wine dataset results

For k=1:

- Both regular and weighted k-NN have identical accuracy (0.7963). This makes sense because with only one neighbor, the weighting has no effect (there's only one point to consider)

For k=3:

- Again, both algorithms show identical performance (0.7407). This suggests that the closest neighbors in the test cases are likely at similar distances, so weighting didn't change the voting outcome

For k=5:

- Here we see our first difference: weighted k-NN (0.7593) outperforms regular k-NN (0.7407). This 1.86% improvement indicates that weighting is starting to have an effect

These results reveal several important insights:

- Weighting becomes more valuable as k increases:** The difference only appeared at $k=5$, suggesting that with more neighbors, giving higher importance to closer points helps classification accuracy.
- Wine dataset is more challenging:** Unlike the perfect scores on Iris, these accuracy values around 75-80% confirm the Wine dataset is more complex and better for algorithm comparison.
- Distance distributions matter:** The improvement at $k=5$ suggests there are cases where some neighbors are significantly closer than others, and weighting these correctly improves classification.

Test higher k values (7, 9, 11) where weighting might show even greater advantages.

Viva Questions

- What is the k-Nearest Neighbors (k-NN) algorithm?
- Is k-NN a supervised or unsupervised learning algorithm? Why?
- What type of problems can k-NN solve?
- What are the advantages of k-NN?
- What are the limitations of k-NN?
- How does k-NN classify a new instance?
- What role does the value of k play in k-NN?
- How do we choose the optimal value of k?
- What happens if k is too small or too large?
- What distance metrics can be used in k-NN? Which one is most commonly used?
- What is the significance of the Euclidean distance in k-NN?
- How does weighted k-NN differ from regular k-NN?
- What real-world applications use k-NN?
- How is k-NN used in medical diagnosis?
- How does k-NN help in recommendation systems like Netflix and Amazon?
- What evaluation metrics are used to measure k-NN's performance?